Lab2 Parser 实现技术报告

> **阅读指南**: 这份报告假设你完全没有编译器、语法分析、编程语言理论方面的背景知识。我们会从最基础的概念开始讲起，逐步深入到你需要理解的每一个细节。即使你上课没有听讲，也应该能通过这份文档理解我们做了什么以及为什么这么做。

第一部分：前置知识 - 理解我们在做什么

1.1 程序的执行流程

当你写了一段代码，比如：

```
let x: i32 = 10;
let y: i32 = 20;
let z: i32 = x + y;
```

计算机并不能直接理解这段文字。程序本身只是一串字符，计算机需要经过一系列"翻译"步骤才能执行它。这个翻译过程就是**编译**。

整个编译流程大致如下：

```
源代码 (source code)
    │
    ▼
┌──────────────────────────────────────┐
│            词法分析 (Lexer)            │
│  将字符序列转换为 Token 序列            │
└──────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────┐
│            语法分析 (Parser)           │  ◀── 我们本次实验的内容
│  将 Token 序列转换为 AST (语法树)      │
└──────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────┐
│            语义分析 (Semantic)         │
│  检查类型、作用域、变量定义等           │
└──────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────┐
│            代码生成 (Codegen)          │
│  生成目标机器码或字节码                 │
└──────────────────────────────────────┘
    │
    ▼
可执行程序
```

1.2 什么是 Token？

Token（词法单元） 是词法分析的产物。

源代码在词法分析器（Lexer）眼中只是一串字符，比如 `let x: i32 = 10;` 。Lexer 的工作是把这些字符切分成一个个有意义的"单词"。

以 `let x: i32 = 10;` 为例，它会被切分成以下 Token：

Token	含义
`let`	关键字"let"

Token	含义
`x`	标识符"x"
`:`	冒号
`i32`	标识符（在这里代表类型名）
`=`	等号
`10`	数字字面量
`;`	分号

你可以把 Token 理解为语言的"词汇"——就像英语中 "I", "love", "you" 是三个单词一样。

1.3 什么是 AST（抽象语法树）？

AST（Abstract Syntax Tree，抽象语法树）是语法分析的产物。

Token 序列虽然已经是有意义的单词了，但单词如何组合、哪个先算、哪个后算，这些"语法规则"还没有被理解。AST 就是用来表达这种语法结构的数据结构。

以表达式 `1 + 2 * 3` 为例：

- 如果不考虑优先级，可能会被理解为 `(1 + 2) * 3 = 9`
- 正确的理解应该是 `1 + (2 * 3) = 7`

用 AST 表示就是：

```
       BinaryOp: +
      /            \
 Literal: 1     BinaryOp: *
           |            |
      Literal: 2    Literal: 3
```

树中的每个**节点**代表一个语法成分，**父子关系**代表"这个节点是那个节点的某一部分"。

1.4 为什么要用树结构？

树结构非常适合表示嵌套和层级关系。考虑一个完整的程序：

```
i32 main() {
    let a: i32 = 10;
    let b: i32 = 20;
    if a > b {
        return a;
    } else {
        return b;
    }
}
```

它的 AST 大致是：

```
Program
└── FuncDef: main() -> i32
    ├── VarDecl: i32 a
    │   └── Initializer: 10
    ├── VarDecl: i32 b
    │   └── Initializer: 20
    └── If
        ├── Condition: a > b
        ├── Then
        │   └── Return: a
        └── Else
            └── Return: b
```

这棵树清晰地展示了程序的层次结构：一棵树根节点是 Program，下面有函数定义，函数定义里有语句，语句又可以分为变量声明、条件语句、返回语句等。

第二部分：本次实验的任务

2.1 实验目标

本次实验要求我们实现一个**语法分析器（Parser）**。

输入: Token 序列（由 Lexer 产生）
输出: AST（抽象语法树）

2.2 原始代码的问题

我们拿到的 `parser.an` 文件是一个**模板**，包含了 7 处 `TODO` 注释。这意味着这个 Parser 只有骨架，没有实际功能。我们的任务就是把这些 TODO 补全。

原始代码的 7 个 TODO 分布在：

1. `parse_enum_def()` - 解析枚举定义
2. `parse_var_decl()` - 解析变量声明
3. `parse_for()` - 解析 for 循环
4. `parse_expr_bp()` - Pratt Parser 核心算法（最难的部分）
5. `parse_prefix_expr()` - 前缀表达式解析
6. `parse_primary_expr()` - 主表达式解析
7. `parse_postfix_expr()` - 后缀表达式解析

第三部分：逐个理解每个 TODO

3.1 `parse_enum_def` - 解析枚举定义

3.1.1 什么是枚举？

枚举（Enum）是一种自定义类型，它限定变量的取值只能是一组固定的选项。

比如定义一个表示颜色的枚举：

```
enum Color {
    Red,
    Green,
    Blue
}
```

这里 `Color` 是枚举类型名，`Red`、`Green`、`Blue` 是它的三个**变体（variant）**。

3.1.2 对比结构体和枚举

结构体定义：

```
struct Point {
    i32 x
    i32 y
}
```

结构体的每个实例包含**多个字段**，每个字段有名字和类型。

枚举定义：

```
enum Color {
    Red,
    Green,
    Blue
}
```

枚举的每个"值"只是一个简单的名字，没有关联数据。

3.1.3 需要实现的代码

参考已有的 `parse_struct_def()` ，我们可以写出 `parse_enum_def()` ：

```
EnumDef parse_enum_def() {
    // 1. 消费关键字 'enum'
    self.consume_keyword(Keyword.Enum)

    // 2. 解析枚举名称（比如 "Color"）
    String name = self.parse_ident()

    // 3. 消费左花括号 '{'
    self.consume_keyword(Keyword.LBrace)

    // 4. 循环解析每个变体（variant）
    Vec<String> variants = Vec<String>.new()
    while not self.peek().is_keyword(Keyword.RBrace) {
        // 解析变体名称（如 "Red"）
        String variant_name = self.parse_ident()
        variants.push(variant_name)

        // 如果遇到逗号，说明后面还有变体，消费逗号继续
        if self.peek().is_keyword(Keyword.Comma) {
            self.consume_keyword(Keyword.Comma)
        }
    }

    // 5. 消费右花括号 '}'
    self.consume_keyword(Keyword.RBrace)

    // 6. 返回枚举定义结构
    return EnumDef(name, variants)
}
```

这段代码的作用：

- 消费 `enum` 关键字
- 读取枚举名称
- 逐个读取变体名称直到遇到 `}`
- 返回一个完整的 `EnumDef` 对象

3.2 `parse_var_decl` - 解析变量声明

3.2.1 YIAN 语言的变量声明语法

在 YIAN 中，变量声明必须使用 `let` 关键字，格式为：

```
let 变量名: 类型 = 初始值;
```

例如：

```
let x: i32 = 10;
let name: str = "hello";
let arr: i32[5] = [1, 2, 3, 4, 5];
```

3.2.2 需要实现的代码

```
Stmt parse_var_decl() {
    // 1. 消费 'let' 关键字
    self.consume_keyword(Keyword.Let)

    // 2. 解析变量名
    String name = self.parse_ident()

    // 3. 消费冒号 ':'
    self.consume_keyword(Keyword.Colon)

    // 4. 解析变量类型（支持数组、指针等复杂类型）
    ASTType type = self.parse_type()

    // 5. 初始化表达式默认为 None（表示没有初始值）
    Option<Expr> init = Option<Expr>.None

    // 6. 如果遇到 '=', 说明有初始值，解析它
    if self.peek().is_keyword(Keyword.Assign) {
        self.consume_keyword(Keyword.Assign)
        init = Option<Expr>.Some(self.parse_expr())
    }

    // 7. 消费分号 ';'
    self.consume_keyword(Keyword.Semicolon)

    // 8. 返回变量声明语句
    return Stmt.VarDecl(VarInfo(name, type), init)
}
```

逐行解释：

代码	作用
`consume_keyword(Keyword.Let)`	确认下一个 Token 是 `let` ， 并移动到下一个位置。如果不是则报错。
`parse_ident()`	读取下一个标识符作为变量名
`consume_keyword(Keyword.Colon)`	确认下一个 Token 是 `:`

代码	作用
`parse_type()`	解析类型，支持 `i32` 、 `str` 、 `i32*` （指针）、 `i32[5]` （数组）等
`Option<Expr> init = Option<Expr>.None`	创建空的可选值，表示"没有初始值"
`if self.peek().is_keyword(Keyword.Assign)`	查看下一个 Token 是否是 `=`
`parse_expr()`	解析等号后面的表达式
`consume_keyword(Keyword.Semicolon)`	确认分号结束

3.3 `parse_for` - 解析 for 循环

3.3.1 YIAN 语言的 for 循环语法

```
for (初始化; 条件; 更新) {
    循环体
}
```

例如：

```
for let i: i32 = 0; i < 10; i += 1 {
    sum += i;
}
```

等价于 C 语言中的：

```
for (int i = 0; i < 10; i += 1) {
    sum += i;
}
```

3.3.2 需要实现的代码

```
Stmt parse_for() {
    // 1. 消费 'for' 关键字
    self.consume_keyword(Keyword.For)

    // 2. 准备三个可选组件：初始化语句、条件、更新表达式
    Option<Stmt*> init = Option<Stmt*>.None
    Option<Expr> condition = Option<Expr>.None
    Option<Expr> update = Option<Expr>.None

    // 3. 如果下一个 token 不是 ';', 说明有初始化语句
    if not self.peek().is_keyword(Keyword.Semicolon) {
        Stmt* init_stmt = dyn Stmt
        *init_stmt = self.parse_stmt()   // parse_stmt() 会消费分号
        init = Option<Stmt*>.Some(init_stmt)
    }

    // 4. 如果下一个 token 不是 ';', 说明有条件表达式
    if not self.peek().is_keyword(Keyword.Semicolon) {
        condition = Option<Expr>.Some(self.parse_expr())
    }

    // 5. 消费分号 ';'
    self.consume_keyword(Keyword.Semicolon)

    // 6. 如果下一个 token 不是 '{', 说明有更新表达式
    if not self.peek().is_keyword(Keyword.LBrace) {
        update = Option<Expr>.Some(self.parse_expr())
    }

    // 7. 解析循环体
    Vec<Stmt> body = self.parse_braced_stmt_block()

    return Stmt.For(init, condition, update, body)
}
```

为什么需要三个 `Option` 类型？

因为 for 循环的每个部分都是可选的。以下这些都是合法的：

```
for (;;) { ... }            // 无限循环
for let i: i32 = 0; ; i++ { ... }  // 没有条件，永远执行
```

3.4 Pratt Parser 算法 - 最核心的部分

这是本次实验**最难**的部分，也是最重要的一部分。

3.4.1 问题引出：为什么需要特殊算法？

考虑表达式： `1 + 2 * 3`

我们希望解析成：

```
      +
     / \
    1   *
       / \
      2   3
```

即先算 `2 * 3 = 6` ，再算 `1 + 6 = 7` 。

但是如果我们简单地"从左到右"解析，会发生什么？

```
步骤1: 读取 1，+ 是运算符
步骤2: 读取 2，* 是运算符
步骤3: 读取 3
```

这样我们得到了 `1 + 2` ，然后呢？ `*` 怎么办？

普通递归下降解析器（naive recursive descent parser）处理这种情况会非常复杂，需要为每种运算符组合写专门的代码。

Pratt Parser 提供了一种优雅的解决方案。

3.4.2 核心概念：绑定力（Binding Power）

Pratt Parser 为每个运算符定义两个数字：**左绑定力（left_bp）和右绑定力（right_bp）**。

运算符	左绑定力	右绑定力	含义
`=`	1	1	赋值（右结合）
`or`	2	3	逻辑或（右结合）
`and`	3	4	逻辑与（右结合）
`==`	7	8	相等比较
`<` `>`	8	9	大小比较
`+` `-`	10	11	加减（左结合）
`*` `/`	11	12	乘除（左结合）

绑定力越大，优先级越高。所以 `*`（左绑定力 11）比 `+`（左绑定力 10）优先级高。

3.4.3 算法思想

Pratt Parser 的核心只有两句话：

1. **先解析左边的表达式**
2. **循环查看右边的运算符，如果它的优先级够高（left_bp >= 当前最低要求），就消费它并继续解析右边**

用一个参数 `min_bp`（minimum binding power）来表示"当前层愿意接受的最低优先级"。数值越大，要求越高。

3.4.4 需要实现的代码

```
Expr parse_expr_bp(u64 min_bp) {
    // 1. 首先解析"前缀表达式"，得到左边的表达式
    Expr left = self.parse_prefix_expr()

    // 2. 进入循环，持续查看是否还有中缀运算符
    loop {
        Token token = self.peek()

        // 3. 尝试将当前 token 转换为中缀运算符信息
        Option<InfixInfo> opt_inf = BinaryOp.from_keyword(t.kw)

        if opt_inf.is_some() {
            InfixInfo inf = opt_inf.unwrap()

            // 4. 如果这个运算符的左绑定力小于 min_bp，说明优先级不够，退出
            if inf.left_bp < min_bp {
                break
            }

            // 5. 优先级足够，消费这个运算符
            self.next()

            // 6. 以右绑定力为新的最低要求，递归解析右边
            Expr* right = dyn Expr
            *right = self.parse_expr_bp(inf.right_bp)

            // 7. 将左右两部分组合成二元表达式
            Expr* new_left = dyn Expr
            *new_left = left
            left = Expr.Binary(new_left, inf.op, right)

            // 8. 继续循环，看后面还有没有运算符
        } else {
            break
        }
    }

    return left
}
```

3.4.5 算法执行示例

以 `1 + 2 * 3` 为例：

第一次调用: `parse_expr_bp(1)` ← 初始调用，最低要求是 1

1. 调用 `parse_prefix_expr()` ，得到 `left = Literal(1)`
2. 循环中看到 `+` ，获取其绑定力 `(10, 11)`
3. `10 >= 1` ，优先级足够，继续
4. 消费 `+` ，递归调用 `parse_expr_bp(11)` 来解析右边

第二次调用: `parse_expr_bp(11)` ← 最低要求是 11

1. 调用 `parse_prefix_expr()` ，得到 `left = Literal(2)`
2. 循环中看到 `*` ，获取其绑定力 `(11, 12)`
3. `11 >= 11` ，优先级足够，继续
4. 消费 `*` ，递归调用 `parse_expr_bp(12)` 来解析右边

第三次调用: `parse_expr_bp(12)` ← 最低要求是 12

1. 调用 `parse_prefix_expr()` ，得到 `left = Literal(3)`
2. 循环中看到没有更多运算符，退出
3. 返回 `Literal(3)`

回到第二次调用：

- 收到右边 `Literal(3)`
- 组合成 `Binary(Literal(2), *, Literal(3))`
- 继续循环，没有更多运算符
- 返回 `Binary(Literal(2), *, Literal(3))`

回到第一次调用：

- 收到右边 `Binary(Literal(2), *, Literal(3))`
- 组合成 `Binary(Literal(1), +, Binary(Literal(2), *, Literal(3)))`
- 继续循环，没有更多运算符
- 返回最终结果

解析结果正是一个优先级正确的二叉树！

3.4.6 为什么使用 `dyn Expr` 和指针？

在 YIAN 语言中， `dyn` 关键字用于动态内存分配。 `Expr*` 表示指向 `Expr` 类型的指针。

当我们需要将一个 `Expr` 值存入另一个复杂数据结构（如 `Binary` 的左右操作数）时，需要用指针来引用它：

```
Expr* new_left = dyn Expr   // 动态分配一个 Expr 指针
*new_left = left            // 把 left 的值存入这块内存
left = Expr.Binary(new_left, inf.op, right)  // 用这个指针构造 Binary
```

3.5 `parse_prefix_expr` - 处理"开头"的运算

3.5.1 什么是前缀表达式？

有些运算符出现在**表达式开头**，而不是中间。这种运算符叫做**前缀运算符**。

在 YIAN 中，前缀运算符包括：

运算符	含义	示例
`-`	算术取负	`-x` 表示 x 的负数
`*`	解引用	`*p` 表示 p 指向的值
`&`	取地址	`&x` 表示 x 的内存地址
`~`	位取反	`~x`
`not`	逻辑非	`not flag`
`dyn`	动态内存分配	`dyn i32`

此外，`true` 和 `false` 也是前缀形式的布尔字面量。

3.5.2 需要实现的代码

```
Expr parse_prefix_expr() {
    Token token = self.peek()

    match token.data {
        Kw as t {
            match t.kw {
                // 负号: -x
                Minus {
                    self.next()
                    Expr* operand = dyn Expr
                    *operand = self.parse_expr_bp(12)  // 用高优先级 12
                    return Expr.Unary(UnaryOp.Neg, operand)
                }
                // 解引用: *p
                Star {
                    self.next()
                    Expr* operand = dyn Expr
                    *operand = self.parse_expr_bp(12)
                    return Expr.Unary(UnaryOp.Deref, operand)
                }
                // 取地址: &x
                Ampersand {
                    self.next()
                    Expr* operand = dyn Expr
                    *operand = self.parse_expr_bp(12)
                    return Expr.Unary(UnaryOp.AddrOf, operand)
                }
                // 位取反: ~x
                Tilde {
                    self.next()
                    Expr* operand = dyn Expr
                    *operand = self.parse_expr_bp(12)
                    return Expr.Unary(UnaryOp.BitNot, operand)
                }
                // 逻辑非: not x
                Not {
                    self.next()
                    Expr* operand = dyn Expr
                    *operand = self.parse_expr_bp(12)
                    return Expr.Unary(UnaryOp.LogicNot, operand)
                }
```

```
            // 动态内存分配: dyn i32 或 dyn i32[3]
            Dyn {
                self.next()
                ASTType type = self.parse_base_type()
                Option<Expr*> array_size = Option<Expr*>.None
                // dyn i32[expr] 的情况
                if self.peek().is_keyword(Keyword.LBracket) {
                    self.consume_keyword(Keyword.LBracket)
                    Expr* size = dyn Expr
                    *size = self.parse_expr()
                    self.consume_keyword(Keyword.RBracket)
                    array_size = Option<Expr*>.Some(size)
                }
                // 处理后续的指针和数组修饰符
                loop {
                    if self.peek().is_keyword(Keyword.LBracket) {
                        // ...
                    } elif self.peek().is_keyword(Keyword.Star) {
                        // ...
                    } else {
                        break
                    }
                }
                return Expr.Dyn(type, array_size)
            }
            // 布尔字面量
            True {
                self.next()
                return Expr.Lit(Literal.Bool(true))
            }
            False {
                self.next()
                return Expr.Lit(Literal.Bool(false))
            }
            _ {}
        }
    }
    _ {}
}

// 如果不是前缀运算符，就当作普通主表达式处理
Expr expr = self.parse_primary_expr()
```

```
    return self.parse_postfix_expr(expr)
}
```

为什么前缀运算符使用绑定力 12？

绑定力 12 是所有运算符中最高的。使用这么高的值可以确保前缀运算符的操作数解析时不会被其他低优先级运算符"截断"。

例如 `-a + b` ：

- `-` 的操作数是 `a + b` 整体
- 如果我们用低绑定力解析 `a + b` ，可能只解析到 `a` 就停了
- 用绑定力 12 解析，可以确保解析完整的最右侧表达式

3.6 `parse_primary_expr` - 最基础的表达式

3.6.1 什么是主表达式？

主表达式（Primary Expression）是不能再分割的最基本表达式单位。包括：

1. **标识符**: `x` ， `foo` ， `bar`
2. **字面量**: `42` ， `3.14` ， `'A'` ， `"hello"` ， `true` ， `false`
3. **括号表达式**: `(a + b)` - 括号内的表达式
4. **数组字面量**: `[1, 2, 3, 4, 5]`

3.6.2 需要实现的代码

```
Expr parse_primary_expr() {
    Token token = self.next()

    match token.data {
        // 标识符，如变量名、函数名
        Ident as t {
            return Expr.Ident(t.name)
        }
        // 字面量，如数字、字符、字符串
        Lit as t {
            return Expr.Lit(Literal.from_token(token))
        }
        Kw as t {
            match t.kw {
                // 括号表达式: (expr)
                LParen {
                    Expr expr = self.parse_expr()
                    self.consume_keyword(Keyword.RParen)
                    return expr
                }
                // 数组字面量: [1, 2, 3]
                LBracket {
                    Vec<Literal> elements = Vec<Literal>.new()
                    while not self.peek().is_keyword(Keyword.RBracket) {
                        elements.push(self.parse_literal_value())
                        if self.peek().is_keyword(Keyword.Comma) {
                            self.consume_keyword(Keyword.Comma)
                        }
                    }
                    self.consume_keyword(Keyword.RBracket)
                    return Expr.Lit(Literal.Array(elements))
                }
                _ { self.error(token) }
            }
        }
        _ { self.error(token) }
    }

    return Expr.Lit(Literal.Int(0))
}
```

逐行解释：

代码	作用
`self.next()`	消费（读取并移动）当前 Token
`Ident as t { return Expr.Ident(t.name) }`	如果是标识符，构造 `Expr.Ident`
`Lit as t { return Expr.Lit(Literal.from_token(token)) }`	如果是字面量，转换为 `Literal` 再构造 `Expr.Lit`
`LParen { ... }`	如果是 `(` ， 递归解析内部表达式， 然后要求 `)` 匹配
`LBracket { ... }`	如果是 `[` ， 循环解析元素直到 `]` ， 构造数组字面量

3.7 `parse_postfix_expr` - 处理"后面"的运算

3.7.1 什么是后缀表达式？

与前缀运算符相对，后缀运算符出现在表达式**后面**。

在 YIAN 中，支持三种后缀运算：

后缀	含义	示例
`()`	函数调用	`foo(1, 2)`
`.field`	字段访问	`point.x`
`[]`	数组下标	`arr[0]`

这些运算可以**链式**出现，例如：

```
a(b)[0].c
| || | || |
| || | || └── 后缀: .c (字段访问)
| || | |└──── 后缀: [0] (下标访问)
| || | └──── 后缀: (b) (函数调用)
| |└─┘──────── 前缀: a (标识符)
```

3.7.2 需要实现的代码

```
Expr parse_postfix_expr(Expr expr) {
    // 持续循环，直到没有后缀为止
    loop {
        Token token = self.peek()
        match token.data {
            Kw as t {
                match t.kw {
                    // 函数调用: ident(args...)
                    LParen {
                        self.next()
                        Vec<Expr> args = Vec<Expr>.new()
                        // 解析参数列表
                        while not self.peek().is_keyword(Keyword.RParen) {
                            args.push(self.parse_expr())
                            if self.peek().is_keyword(Keyword.Comma) {
                                self.consume_keyword(Keyword.Comma)
                            }
                        }
                        self.consume_keyword(Keyword.RParen)
                        // 确保被调用的是标识符
                        match expr {
                            Ident as id {
                                expr = Expr.Call(id.name, args)
                            }
                            _ { self.error(token) }
                        }
                    }
                    // 字段访问: expr.field
                    Dot {
                        self.next()
                        String field = self.parse_ident()
                        Expr* base = dyn Expr
                        *base = expr
                        expr = Expr.FieldAccess(base, field)
                    }
                    // 下标访问: expr[index]
                    LBracket {
                        self.next()
                        Expr* index = dyn Expr
                        *index = self.parse_expr()
                        self.consume_keyword(Keyword.RBracket)
```

```
                        Expr* base = dyn Expr
                        *base = expr
                        expr = Expr.Binary(base, BinaryOp.Index, index)
                    }
                    _ { break }
                }
            }
            _ { break }
        }
    }

    return expr
}
```

链式解析的过程：

以 `a(b)[0]` 为例：

```
初始: expr = Ident("a")

循环第1次:
- 看到 LParen '('
- 解析参数 [Ident("b")]
- 组合: expr = Call("a", [Ident("b")])

循环第2次:
- 看到 LBracket '['
- 解析索引 Literal(0)
- 组合: expr = Binary(Call("a", [Ident("b")]), Index, Literal(0))

循环第3次:
- 看到 Dot '.'
- 解析字段名 "c"
- 组合: expr = FieldAccess(Binary(...), "c")

循环第4次:
- 没有更多后缀，退出
- 返回最终表达式
```

第四部分：完整的数据流

4.1 从 Token 到 AST 的完整流程

```
Token 序列
    │
    ▼
parse()                     [入口，解析整个程序]
    │
    ├─► parse_struct_def() [解析结构体定义]
    ├─► parse_enum_def()   [解析枚举定义]
    └─► parse_func_def()   [解析函数定义]
            │
            ├─► parse_type()         [解析返回类型]
            ├─► parse_ident()        [解析函数名]
            ├─► parse_type()         [解析参数类型]
            ├─► parse_ident()        [解析参数名]
            │
            ▼
        parse_braced_stmt_block()    [解析函数体]
            │
            ▼
        parse_stmt()                 [解析语句]
            │
            ├─► parse_var_decl()     [变量声明]
            ├─► parse_if()           [if 语句]
            ├─► parse_for()          [for 循环]
            ├─► parse_return()       [return 语句]
            └─► parse_expr_stmt()    [表达式语句]
                    │
                    ▼
                parse_expr()         [解析表达式]
                    │
                    ▼
                parse_expr_bp(1)     [Pratt Parser 核心]
                        │
                        ├─► parse_prefix_expr()     [前缀运算符]
                        │       ├─► parse_primary_expr() [基础表达式]
                        │       └─► parse_postfix_expr() [后缀运算符]
                        │
                        └─► [中缀运算符循环]
```

4.2 辅助函数的作用

函数	作用
`peek()`	查看当前 Token 但不消费它
`peek_n(n)`	查看后面第 n 个 Token
`next()`	消费并返回当前 Token
`consume_keyword(kw)`	确保下一个 Token 是指定关键字，否则报错
`parse_ident()`	解析标识符
`parse_type()`	解析类型（支持数组和指针）
`parse_integer()`	解析整数字面量

第五部分：测试验证

5.1 测试用例概览

测试文件	重点内容
01.an	基本类型（i8-i64, u8-u64, f32, f64, char, bool, str）
02.an	函数定义、控制流（if, while, for, return）
03.an	表达式运算（算术、比较、逻辑）
04.an	复杂表达式（优先级、结合性）
05.an	结构体、枚举、模式匹配（match）
06.an	数组、指针、动态内存分配（dyn）

5.2 验证方法

```
# 编译
scripts/yian_compiler.py labs/lab2/parse

# 运行
./tests/yian_workspace/bin/out

# 对比输出
diff labs/lab2/output/ labs/lab2/test_results/
```

所有 6 个测试用例均通过，输出与参考答案完全一致。

第六部分：总结

6.1 我们做了什么

本次实验我们实现了一个完整的递归下降语法分析器，支持：

1. **类型定义**: 结构体、枚举、函数定义
2. **变量声明**: `let name: type = value;`
3. **控制流**: if/else, while, for, loop, match
4. **表达式**: 算术运算、比较运算、逻辑运算、赋值
5. **优先级处理**: 通过 Pratt Parser 算法正确处理运算符优先级
6. **后缀运算**: 函数调用、字段访问、数组下标
7. **动态内存**: `dyn` 关键字

6.2 Pratt Parser 的核心思想

1. 每个中缀运算符有 `(left_bp, right_bp)` 两个优先级
2. 通过 `min_bp` 参数控制"当前层愿意接受的最低优先级"
3. 如果 `left_bp < min_bp` ，说明该运算符优先级不足，交给外层处理
4. 递归地解析左右两侧，确保正确的结合性

6.3 后续学习建议

- 了解语义分析（Semantic Analysis）阶段做什么

- 了解符号表（Symbol Table）的作用
- 了解类型检查（Type Checking）的基本原理
- 了解代码生成（Code Generation）的基本流程